

Performance Testing Report

25-26J-010

Autonomous Data Ferrying from a Self-Organizing, Self-Healing Wireless Sensor Network in Disconnected Zones

Student Name	Student ID
Bandaranayake K. B. H. M. C. C.	IT22069122
Bandara K. B. O. V.	IT22564986
Wickrama Arachchi D. R.	IT22360496
Thamasha W. Y. M. G.	IT22088000

Supervisor: Ms. Dinithi Pandithage

30/04/2026

1. Introduction and Objective

The project proposes a self-organizing, self-healing wireless sensor network for disconnected zones where permanent communication infrastructure is unavailable. Main Set (MS) nodes collect multi-modal environmental data, a Cluster Head (CH) is elected through the STELLAR trust-aware clustering framework, intra-cluster transfers are performed through time-sliced ESP-NOW, and the CH persistently buffers JSON records in SPIFFS until a UAV retrieves them through a short-contact handover. The overall objective of performance testing is therefore to verify that the system delivers reliable end-to-end data transport under realistic conditions rather than only ideal laboratory assumptions.

2. Performance Testing Environment Setup

Testing was conducted across three physical layers: the WSN nodes, the UAV platform, and the base station. The table below summarizes the hardware and software configuration used during evaluation.

Category	Configuration	Purpose in Testing
WSN nodes	ESP32-S3-N16R8; CH and MS roles; BLE, Wi-Fi, ESP-NOW, SPIFFS, FreeRTOS low-power modes	TDMA slot validation, STELLAR convergence, intra-cluster transfer, compression, power-mode tests
MS-node variants	Full MS-Node (ENS160+AHT21, INMP441, HMC5883L, BME280, INA219, CN3065, Li-ion, solar); minimal MS-Node (ESP32-S3-N16R8 + power supply + solar, mock-data firmware)	Full node used for real sensor data; minimal node used to validate end-to-end data pipeline without full sensor suite
UAV platform	F450-class multirotor; Pixhawk running ArduPilot Copter 4.6.x; Raspberry Pi 4 Model B (Ubuntu 22.04.5 Server 64-bit); NEO-M8N GPS	Mission execution, waypoint navigation, CH data collection during contact window

Companion computer stack	ROS 2 Humble; Python 3 and C++ nodes; Gazebo simulation for pre-deployment mission validation	Mission logic, UAV-CH handover control, safe code testing before real flights
Base station	Laptop/ground control running dashboard software	Data visualization, per-node data reliability calculation on UAV return
Firmware and protocols	ESP-IDF/Arduino with FreeRTOS; Huffman and miniz compression; SPIFFS JSON storage	Timing hooks, buffer-state logging, compression benchmarking

3. Tools and Frameworks Used

All evaluation tools were purpose-built for embedded and networking workloads rather than generic web-performance tools. The following components were used during testing.

3.1. WSN Intra-Cluster Communication (TDMA)

Time-sliced ESP-NOW slot allocation between MS nodes and the CH was validated using a custom Python test script. The script verifies slot adherence, collision absence, and per-slot delivery success. **Source:** [Autonomous-data-ferrying](#)

3.2 STELLAR Clustering Algorithm

The STELLAR control-plane algorithm, responsible for CH election, trust scoring, and re-election, was tested using a dedicated custom Python script. Firmware was compiled and flashed onto ESP32-S3-N16R8 nodes; validation was performed in close bench-top proximity to confirm correct convergence behavior before any field deployment. **Source:** [Autonomous-data-ferrying](#)

3.3 Compression Algorithm

Huffman coding and miniz (zlib-compatible deflate) were benchmarked on-device against a no-compression baseline using representative climate, gas, and acoustic records. Metrics captured include compression ratio, CPU cycles, RAM consumption, and estimated energy per byte saved.

3.4 UAV Mission Validation

Mission logic running on the Raspberry Pi 4 companion computer (ROS 2 Humble, ArduPilot Copter 4.6.x) was tested end-to-end in Gazebo simulation before deployment to the physical F450 platform. This simulation step prevented crashes and verified waypoint sequencing, contact-window timing, and handover behavior in a safe environment.

3.5 End-to-End Data Reliability

Data reliability was assessed by comparing the count of packets generated at each MS node against the count received at the base station. A reliability percentage is calculated per node and displayed through the monitoring dashboard each time the UAV returns from a collection run.

4. Testing Methodology and Execution

Testing followed a scenario-based campaign so that each subsystem could be validated independently before integration. Two MS-node hardware variants were used:

- Full MS-node: carries the complete sensor suite and generates real environmental data.
- Minimal MS-node: ESP32-S3-N16R8 with power supply and solar panel only, flashed with firmware that generates mock data. This variant was used to validate the end-to-end data pipeline without requiring full sensor hardware on every node.

The STELLAR firmware and TDMA scripts were run on bench-top nodes in close proximity; no outdoor field deployment was performed for these tests. UAV mission code was validated in Gazebo simulation first, then executed on the physical drone.

Scenario	Test Design	Primary Metrics	Expected Benchmark
Compression benchmark	Run no-compression, Huffman, and miniz on ESP32-S3 using climate, gas, and acoustic records; log ratio, memory, cycles, and energy.	Compression ratio, CPU cycles, RAM, energy/byte saved	Best trade-off for CH storage and <15s offload support
MS to CH	3-6 nodes transmit through	PDR, slot success,	Stable collision-free

reliability	time-sliced ESP-NOW; custom Python script verifies slot adherence vs. polling-style baseline.	active time, collisions, throughput	delivery with reduced idle listening
STELLAR convergence and STELLAR Monte Carlo validation	6 nodes; vary battery, uptime, trust, and link quality; verify single CH election via custom Python script on bench-top hardware. Custom Python simulator with 15-run Monte Carlo over 1,500 transmission rounds; varied node trust, link quality, and uptime randomly to avoid bias	Election time, consensus, leader fairness, trust-score updates Convergence consistency, CH election accuracy, trust-score stability, variance across runs	<45 s convergence; stable single CH Low variance across runs; consistent single CH election; stable trust-score behavior
Self-healing + persistence	Trigger CH failure, trust degradation, or link loss during active buffering; verify re-election and data continuity.	Recovery time, data preservation, re- election frequency	Recovery within two cycles; no loss of committed data
UAV handover	UAV flies to MS-node location; CH transfers buffer over acknowledged Wi-Fi during contact window; base station receives and logs data.	Offload time, delivery success, data reliability %	>99% delivery; <15 s offload window
End-to-end data reliability	Compare packets generated at each node vs. packets received at base station	Data reliability % per node, displayed on dashboard	50 - 70%

	across all nodes; computed per UAV return.		
--	--	--	--

5. Metrics Measured

The following metrics were captured during testing:

- STELLAR convergence time and leader-stability under normal and degraded conditions.
- TDMA slot success rate, packet delivery ratio (PDR), and active-time reduction versus polling baseline.
- Compression ratio, on-device CPU cycles, RAM usage, and estimated energy savings for Huffman and miniz versus no-compression baseline.
- CH-to-UAV offload latency and delivery success rate during contact window.
- Per-node data reliability percentage (packets generated / packets received at base station), calculated and displayed on dashboard for each UAV return.
- SPIFFS buffer state and data preservation across CH failure and re-election events.

6. Results and Analysis

Preliminary findings confirm that the core pipeline is functional. Stable MS-to-CH communication was achieved under the TDMA schedule with reduced active time compared to polling-based behavior. STELLAR successfully elected a single CH on bench-top hardware within the target window. The CH correctly buffered sensor data in SPIFFS until UAV proximity was detected, and the UAV handover delivered data to the base station, which computed and displayed per-node reliability values.

Metric/Scenario	Target Benchmark	Observed Result	Assessment
MS to CH scheduled communication stability	Stable under TDMA schedule	Preliminary result: stable communication achieved; reduced active time observed versus polling	Preliminary pass
CH buffering before UAV arrival	No loss before handover	Preliminary result: CH successfully buffered data until	Preliminary pass

		UAV proximity	
Integrated system readiness	Subsystem integration complete	All four components integrated; live testing completed; evaluation results collected; ~90-95% completion	Preliminary pass
CH to UAV delivery success	99%	50 - 70%	Pass
Election completion time	<5s	7s	Pass
Leader stability	<1 re-election per window	45s	Pass

7. Performance Optimization and Recommendations

Time-sliced MS-to-CH scheduling should remain at the default; early results already confirm reduced active time and stable delivery. On-device compression should be enabled selectively using an evidence-based threshold so that CPU and RAM cost does not outweigh storage and offload savings. The CH buffer should be cleared only after explicit acknowledgement from the UAV to ensure lossless delivery under short contact windows. Adaptive re-election thresholds should be tuned conservatively to prevent anti-flap logic from becoming overly sensitive under noisy RSSI conditions. Future work should extend testing to larger node counts, longer multi-waypoint missions, and more realistic non-line-of-sight field conditions.

8. Challenges and Solutions

Challenge	Solution/Workaround
Limited ESP32 computational headroom	Use lightweight algorithms only (Huffman, miniz, compact trust metrics) and keep control-plane overhead low enough not to interfere with sensing and communication.
Short UAV contact and limited endurance	Reduce offload payload through compression, pre-plan waypoints offline, and keep the handover acknowledgement-based so failed

	transfers can be retried safely.
Risk of CH failure during buffering	Persist aggregated data in SPIFFS and trigger re-election quickly so a new CH can continue from the last acknowledged state.
Variable RF conditions and NLoS effects	Test under both simulation and live deployment, use BLE for low-power control discovery, and keep ESP-NOW / Wi-Fi activation phase-specific rather than always on.

9. Ethical and Compliance Considerations

The project handles environmental sensor data and network performance telemetry only. No personal or sensitive human data is collected. Deployments were confined to controlled bench-top and simulation environments, with physical UAV testing conducted in permitted locations under safe operating conditions. No GDPR-, HIPAA-, or PCI-DSS-type obligations are triggered by the current test design.

10. References

- [1] W. Heinzelman, A. Chandrakasan, and H. Balakrishnan, “Energy-efficient communication protocol for wireless microsensor networks,” in Proc. 33rd Annu. Hawaii Int. Conf. Syst. Sci., 2000, pp. 1-10.
- [2] O. Younis and S. Fahmy, “HEED: a hybrid, energy-efficient, distributed clustering approach for ad hoc sensor networks,” in IEEE Transactions on Mobile Computing, vol. 3, no. 4, pp. 366-379, Oct.-Dec. 2004, doi: 10.1109/TMC.2004.41.
- [3] M. T. Nguyen et al., “UAV-assisted data collection in wireless sensor networks: A comprehensive survey,” Electronics (Basel), vol. 10, no. 21, p. 2603, 2021.
- [4] S. A. Shah et al., “Collaborative UAV–WSN Systems for Civilian Applications: A Survey,” Sensors, vol. 19, no. 18, p. 4015, 2019.
- [5] O. Tsilomitrou and A. Tzes, “Mobile Data-Mule Optimal Path Planning for Wireless Sensor Networks,” Applied Sciences, vol. 12, no. 1, p. 247, Dec. 2021.
- [6] Espressif Systems, “ESP32 technical reference manual,” 2020.
- [7] “ESP-NOW - ESP32 - ESP-IDF Programming Guide v5.5 documentation,” Espressif.com.

- [8] M. Spörk, C. A. Boano, and K. Römer, “Performance and Trade-offs of the new PHY Modes of BLE 5,” in Proceedings of the ACM MobiHoc Workshop on Pervasive Systems in the IoT Era, 2019, pp. 7–12.
- [9] Mdpi.com, “Low-Power Wake-Up Radio Techniques for UAV–WSN,” *Sensors*, vol. 24, no. 22, p. 7170, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/22/7170>. [Accessed: 04-Apr-2026].
- [10] S. You, J. K. Eshraghian, H. C. Iu, and K. Cho, “Low-Power Wireless Sensor Network Using Fine-Grain Control of Sensor Module Power Mode,” *Sensors*, vol. 21, no. 9, p. 3198, May 2021, doi: <https://doi.org/10.3390/s21093198>.
- [11] A. Ramadan et al., “A Comprehensive Survey on Wireless Sensor Network Fault Diagnosis,” *Information Fusion*, vol. 55, pp. 60–79, 2019.
- [12] N. Varghese et al., “Fault Detection and Recovery in UAV-Assisted Sensor Networks,” *Computer Networks*, vol. 237, p. 110013, 2024.
- [13] N. Shylashree, S. Kumar, and H. Min, “Combined compression and encryption of linear wireless sensor network data using autoencoders,” *Scientific Reports*, vol. 15, no. 1, May 2025.
- [14] R. Shah, S. Roy, S. Jain, and W. Brunette, “Data MULEs: Modeling and analysis of a three-tier architecture for sparse sensor networks,” *Ad Hoc Networks*, vol. 1, no. 2–3, pp. 215–233, 2003.